

React 손코딩 훈련 핸드북

칸반 보드로 배우는 React

도완 × Claude (시니어 팀장)

2026-10-01 기준

서문 — 이 문서는 무엇인가

왜 시작했나

진행 방식

프로젝트: 칸반 보드 (Trello 미니 클론)

9단계 로드맵

이 문서 읽는 법

전체 흐름 한눈에

단계별로 무엇이 나아졌나 (요약)

Lesson 1 — 정적 보드 렌더링

1. 과제

2. 전체 그림

3. 최종 코드

4. 대화에서 나온 질문과 답

5. 삽질 기록

6. 팁

7. 면접 문답 (실제 답변 + 교정)

Lesson 2 — 카드 CRUD

1. 과제

1.5 1단계 대비 나아진 것

2. 전체 그림: 데이터는 아래로, 사건은 위로

3. 최종 코드

4. 흐름 추적 예시: “Done”(id 300)의 “살기”(id 5) 삭제

5. 대화에서 나온 질문과 답

6. 삽질 기록

7. 팁

8. 면접 문답 (실제 답변 + 교정)

Lesson 3 — useReducer + Context 리팩터

1. 과제

1.5 2단계 대비 나아진 것

2. 전체 그림

3. 최종 코드

4. 리듀서는 그냥 함수다 — React 없이 돌려보기

5. 대화에서 나온 질문과 답

6. 삽질 기록

7. 팁

8. 면접 문답 (실제 답변 + 교정)

부록 A — 핵심 개념 한눈에

부록 B — 면접 질문 모음 (답할 수 있어야 하는 것)

1단계

2단계

3단계

부록 C — 작업 습관 (팁 모음)

서문 — 이 문서는 무엇인가

왜 시작했나

바이브 코딩에만 의존해서 혼자 손코딩도, 면접에서 설명도 못 하는 상태를 끊기 위해 2026-09-30에 시작했다. 목표는 동작하는 결과물이 아니라 “왜 이렇게 했는지 설명할 수 있는 실력”이다.

진행 방식

- **Claude = 시니어 프론트 팀장.** 단계별 스펙 작성, 코드 리뷰, 면접 질문, 방향 제시. 코드는 원칙적으로 쓰지 않고, 막혔을 때 “틀”이나 “한 줄”만.
- **도완 = 구현자.** `kanban/src/` 아래 코드를 직접 작성. 리뷰 후 면접 질문에 답하고, 교정받는다.
- **문서는 전부 Claude가** 대화 기반으로 정리. 이 PDF가 그 결과물이다.

스펙 작성 → 구현 → "리뷰해줘" → 리뷰 + 면접 질문 → 답변 → 교정 → 교재/회고 정리 → 다음 단계

프로젝트: 칸반 보드 (Trello 미니 클론)

Vite + React 19 + TypeScript 6, CSS Modules. 상태 라이브러리 없음, 라우터 없음, DnD 라이브러리 없음. 전부 손으로. 저장소: github.com/dowankim1024/frontend_practice

왜 칸반인가: 외부 API 없이 시작 가능하고, 상태 모델링·불변 업데이트·useReducer·Context·DnD·undo/redo·낙관적 업데이트까지 면접 단골 주제가 전부 자연스럽게 나온다.

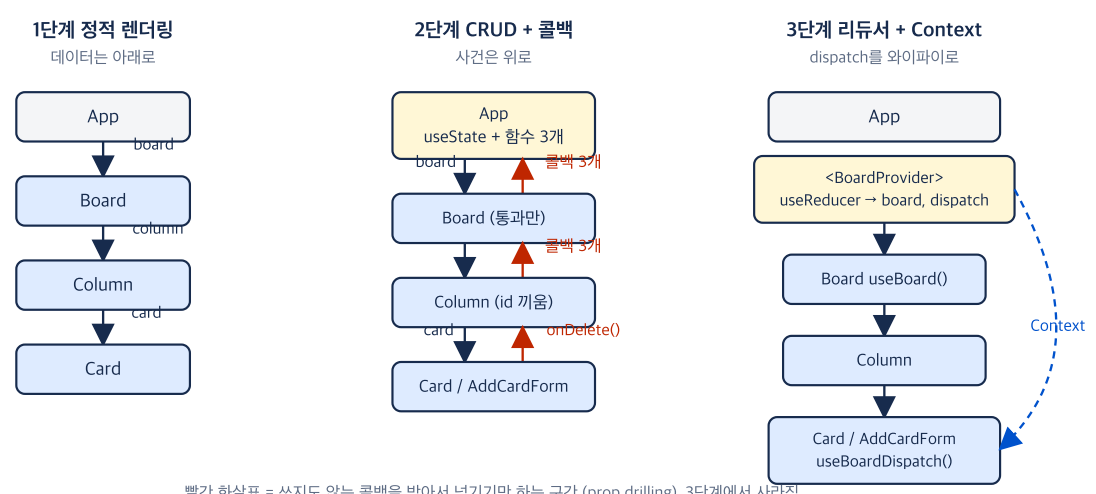
9단계 로드맵

#	주제	핵심 개념	상태
1	정적 보드 렌더링	컴포넌트 분리, props, key, 타입 설계	완료 (09-30)
2	카드 CRUD	제어 컴포넌트, 불변 업데이트, 상태 위치, 콜백 체인	완료 (09-30)
3	useReducer + Context	액션 설계, 리듀서 순수성, Context 분리	완료 (10-01)
4	localStorage 영속화	커스텀 훅, useEffect 의존성, lazy init	
5	드래그앤드롭 (네이티브)	이벤트 흐름, dataTransfer, 드롭 위치 계산	
6	undo/redo	히스토리 상태 설계, 리듀서 합성	
7	mock API + 낙관적 업데이트	msw, 로딩/에러, 롤백	
8	테스트	Vitest, Testing Library, 리듀서 단위 테스트	
9	성능	Profiler, memo/useCallback을 필요한 곳에만	

이 문서 읽는 법

각 장은 과제 → 전체 그림 → 최종 코드 전문 + 해설 → 대화에서 나온 질문과 답 → 삽질 → 팁 → 면접 문답 순서다. 코드는 그 단계가 끝난 시점의 실제 파일 내용이다. 단계마다 코드가 어떻게 변해가는지 비교하면서 읽으면 “왜 리팩터했나”가 보인다.

전체 흐름 한눈에



1~3단계 구조 변화

단계별로 무엇이 나아졌나 (요약)

단계	추가된 능력	치른 비용	그 비용을 푸는 단계
1	타입이 잡힌 데이터를 화면에 그림	—	—
2	카드 추가/삭제/수정. 사건이 콜백으로 위로 올라감	Board 가 쓰지 않는 콜백 3개를 통과 (prop drilling). 변경 로직이 App 에 묶임	3
3	변경 로직이 순수 함수(리듀서)로 분리. dispatch 하나를 Context로 배달. 중간층이 콜백을 모름	Card 가 columnId 를 알아야 함. 리렌더 범위는 아직 그대로	9
4~9	(예정) 영속화 → DnD → undo → 서버 동기화 → 테스트 → 성능		

Lesson 1 — 정적 보드 렌더링

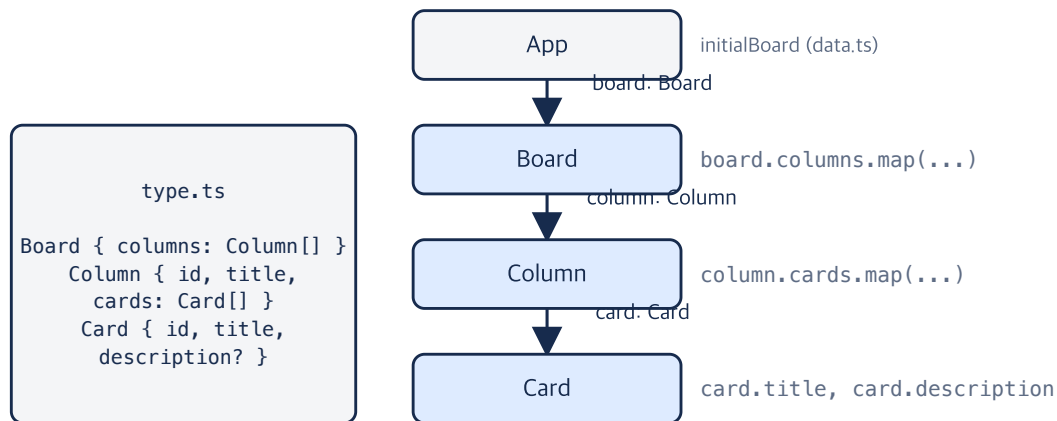
이 문서는 1단계 전체를 나중에 혼자 읽어도 이해되도록 쓴 교재다. 과제 → 최종 코드 → 줄 단위 해설 → 대화에서 나온 질문 → 삽질 → 면접 문답 순서.

1. 과제

하드코딩된 데이터로 컬럼 3개와 카드를 화면에 그린다. 상태 변경 없음. 요구: `type.ts` 에 `Card / Column / Board` 타입, `data.ts` 에 초기 데이터, `Board / Column / Card` 컴포넌트 3개 + CSS Modules, `App` 은 `Board` 에 데이터만 넘김.

2. 전체 그림

1단계: 데이터는 props로만 아래로 흐른다. 각 컴포넌트는 받은 것을 그리기만 한다.



도메인 타입 = 데이터 모양
Props 타입(BoardProps 등)은 각 컴포넌트 파일 안에. 콜백 같은 '데이터 아닌 것'이 들어갈 자리.

1단계 컴포넌트 트리과 데이터 흐름

데이터는 위에서 아래로 props로만 흐른다. 각 컴포넌트는 받은 것을 그리기만 한다.

이 단계가 만드는 출발점

아직 “나아진 것”은 없다. 대신 뒤 단계가 전부 이 위에 쌓이는 **토대 세 개**를 만든다.

토대	뒤에서 어떻게 쓰이나
도메인 타입 Board/Column/Card	2단계 상태 타입, 3단계 리듀서 시그니처, 4단계 저장 단위
컴포넌트 3개가 같은 패턴	2단계에서 콜백을 끼울 자리, 3단계에서 <code>useBoard()</code> 로 바꿀 자리가 명확
key = id	2단계 삭제, 5단계 순서 변경에서 상태가 엉뚱한 카드에 붙는 걸 방지

파일 구조:

```
src/
  type.ts           도메인 타입 (데이터 모양)
  data.ts           초기 데이터
  App.tsx           상태 루트 (지금은 데이터 넘기기만)
  components/
    Board.tsx + Board.module.css
    Column.tsx + Column.module.css
    Card.tsx + Card.module.css
```

3. 최종 코드

src/type.ts

```
export interface Card {
  id: string
  title: string
  description?: string
}
export interface Column {
  id: string
  title: string
  cards: Card[]
}
export interface Board {
  columns: Column[]
}
```

해설 - Card.description? 만 옵셔널. 설명은 정말 없을 수 있는 정보. - Column.cards, Board.columns 는 필수. 비어 있을 수는 있어도 ([]) 없을 수는 없다. → 질문 Q1 참고. - Board 는 지금은 컬럼 묶음일 뿐이지만, 4단계(localStorage 저장 단위)·6단계(undo 단위)에서 상태의 루트가 된다. - 데이터 모델은 **중첩 구조**(컬럼이 카드 배열을 품음)를 선택. 정규화(카드를 별도 맵으로, 컬럼은 cardIds만)보다 단순. 5단계 DnD에서 카드 이동 시 두 컬럼 배열을 동시에 갱신해야 하는 비용은 그때 체감.

src/data.ts

```

import type { Board } from "./type";

export const initialBoard: Board = {
  columns: [
    {
      id: "100",
      title: "To Do",
      cards: [
        {
          id: "1",
          title: "공부하기"
        },
        {
          id: "2",
          title: "잠자기",
          description: "꿀잠잘것"
        }
      ]
    },
    {
      id: "200",
      title: "In Progress",
      cards: [
        {
          id: "3",
          title: "놀이"
        },
        {
          id: "4",
          title: "먹기",
          description: "먹다지침"
        }
      ]
    },
    {
      id: "300",
      title: "Done",
      cards: [
        {
          id: "5",
          title: "살기"
        },
        {
          id: "6",
          title: "태어나기",
          description: "죽지못해태어나기"
        }
      ]
    }
  ]
};

```

해설 - : Board 를 붙이는 순간 모양이 틀리면 컴파일러가 잡아준다. 실제로 처음엔 배열을 그대로 넣어 Property 'columns' is missing 에러가 났고, { columns: [...] } 로 감싸서 해결. - id는 전부 유일해야 한다. 처음에 카드 두 개가 "5" 로 중복이었음 → key 경고와 2단계 삭제 오작동의 원인이 되므로 수정. - 이름은 data 가 아니라 initialBoard. 나중에 localStorage에서 읽은 보드와 구분돼야 한다.

src/components/Card.tsx

```
import type { Card as CardData } from "../type"
import styles from "./Card.module.css"

interface CardProps {
  card: CardData
}

export function Card({ card }: CardProps) {
  return (
    <div className={styles.card}>
      <h3 className={styles.title}>{card.title}</h3>
      {card.description} && <p className={styles.description}>{card.description}</p>
    </div>
  )
}
```

해설 - `import type { Card as CardData } from "../type"` — 타입 `Card`와 컴포넌트 `Card`가 같은 이름이라 별칭. TS는 타입/값 네임스페이스가 달라 컴파일은 되지만 사람이 헷갈린다. - `interface CardProps { card: CardData }` — **props 타입은 컴포넌트 파일 안에**. `type.ts`의 `Card`는 데이터 모양, `CardProps`는 이 컴포넌트의 입력 계약. 2단계에서 `onDelete` 같은 콜백이 여기 추가되는데, 그건 데이터가 아니다. - `{ card }: CardProps` — 매개변수 구조분해. 중괄호 안 이름은 **타입에 실제로 있는 키**여야 한다. `{ props }`라고 쓰면 그런 키가 없어 에러. - 카드를 통째로 받는 이유: 필드가 늘어도 부모 코드가 안 바뀌고, 5단계에서 `card.id`가 어차피 필요. - `{card.description} && <p>...</p>}` — 있을 때만 그린다. 문자열이라 안전. 숫자였다면 `0`이 찍히는 함정(Q3). - 감싸는 `div`가 있어야 CSS 클래스를 붙일 상자가 생긴다. `Fragment(<>)`는 DOM을 안 만든다.

src/components/Column.tsx

```
import type { Column as ColumnData } from "../type"
import { Card } from "./Card"
import styles from "./Column.module.css"
interface ColumnProps {
  column: ColumnData
}
export function Column({ column }: ColumnProps) {
  return (
    <div className={styles.column}>
      <h2 className={styles.title}>{column.title}</h2>
      {column.cards.map((card) => <Card key={card.id} card={card} />)}
    </div>
  )
}
```

해설 - `Card`와 완전히 같은 패턴. 별칭 `import` → `props` 인터페이스 → 구조분해 → `JSX`. - `column.cards.map((card) => <Card key={card.id} card={card} />)` — **key는 map이 반환하는 요소에**. `Card` 안쪽 `div`에 붙이면 배열 밖이라 `React`가 못 본다. - `key`는 `props`가 아니다. `Card` 안에서 `props.key`로 못 꺼낸다.

src/components/Board.tsx

```
import type { Board as BoardData } from "../type"
import { Column } from "../Column"
import styles from "../Board.module.css"
interface BoardProps {
  board: BoardData
}
export function Board({ board }: BoardProps) {
  return (
    <div className={styles.board}>
      {board.columns.map((column) => <Column key={column.id} column={column} />)}
    </div>
  )
}
```

src/App.tsx

```
import { Board } from "../components/Board"
import { initialBoard } from "../data"
function App() {
  return (<div>
    <h1>Kanban</h1>
    <Board board={initialBoard} />
  </div>
)
}

export default App
```

CSS

```
/* Board.module.css - 컬럼들을 가로로 */
/* 보드 전체. 안에 있는 컬럼들을 가로로 나열한다. */
.board {
  display: flex;
  flex-direction: row;
  align-items: flex-start;
  gap: 12px;

  padding: 16px;
  overflow-x: auto;
  min-height: 100vh;
}
```

```

/* Column.module.css - 카드들을 세로로 */
/* 컬럼 하나. 안에 있는 카드들을 세로로 쌓는다. */
.column {
  display: flex;
  flex-direction: column;
  gap: 8px;

  width: 272px;
  flex-shrink: 0;
  padding: 12px;
  border-radius: 8px;
  background: #ebecf0;
}

.title {
  margin: 0 0 4px;
  font-size: 14px;
  font-weight: 700;
  color: #172b4d;
}

```

```

/* Card.module.css */
.card {
  background: #fff;
  border-radius: 6px;
  padding: 8px 12px;
  box-shadow: 0 1px 2px rgba(9, 30, 66, 0.25);
}

.title {
  margin: 0;
  font-size: 14px;
  font-weight: 600;
}

.description {
  margin: 4px 0 0;
  font-size: 12px;
  color: #5e6c84;
}

```

해설 - 정렬 방향은 자식이 아니라 부모에. 카드를 세로로 쌓으려면 카드가 아니라 카드를 담은 `.column`에 `flex-direction: column`. 컬럼을 가로로 놓으려면 `.board`에 `row - gap`으로 자식 간 간격. 각 자식에 `margin` 주는 것보다 깔끔. - CSS Modules: 파일명이 `*.module.css`면 Vite가 클래스명을 고유하게 바꿔준다. `styles.card`로 접근.

4. 대화에서 나온 질문과 답

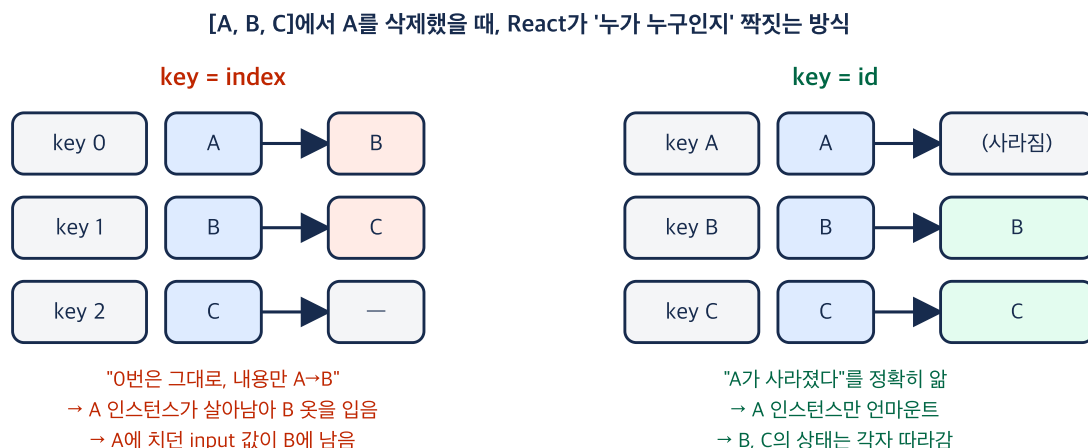
Q1. 카드가 없는 컬럼이 있을 수 있으니 `cards?`로 해야 하지 않나? 아니다. “카드 0개”는 `cards: []`로 이미 표현된다. ?는 “필드 자체가 없을 수 있다(undefined)”는 뜻이라 상태를 하나 더 만드는 것. `[]`.`map()`은 조용히 아무것도 안 그리지만 `undefined.map()`은 터진다. ?는 오류를 막는 게 아니라 방어 코드를 강제한다. → “비어 있을 수 있나”면 빈 값, “없을 수 있나”면 ?.

Q2. Board의 역할이 뭔가? 컬럼 묶음 이상이 아닌데. 지금은 그렇다. 하지만 4단계에서 “무엇을 저장하나”의 답, 6단계에서 “무엇으로 되돌리나”의 답이 `Board` 하나가 된다. 상태 루트 자리를 미리 잡아두는 것.

Q3. “props 타입을 만들라”는 게 `type.ts` 이름을 `CardProps` 로 바꾸라는 건가? 아니다. 도메인 타입(데이터 모양)과 props 타입(컴포넌트 입력 계약)은 다른 개념. props에는 콜백처럼 데이터가 아닌 것도 들어간다. `type.ts` 는 그대로 두고 `CardProps` 는 `Card.tsx` 안에 따로 선언.

Q4. 정렬 방향을 어디에 쓰나? 자식이 아니라 부모 컨테이너에. 위 CSS 해설 참고.

Q5. index를 key로 쓰면 “리렌더가 안 된다”는 건가?



index key vs id key

아니다. 리렌더는 되는데 **잘못된 짝으로** 된다. `[A, B, C]` 에서 A 삭제 → `[B, C]` . index key면 React는 “0번은 그대로 있고 내용만 A→B로 바뀐”으로 본다. A 인스턴스가 살아남아 B 옷을 입고, A에 치던 input 값이 B에 남는다. id key면 “A가 사라졌다”를 정확히 알고 A만 언마운트.

5. 삽질 기록

증상	원인	해결
<code>cards?</code> , <code>columns?</code> 로 선언	“비어 있음”과 “없음” 혼동	필수로 변경
<code>data.ts</code> 최상위가 배열	<code>Board</code> 는 <code>{ columns }</code> 객체	감싸기. 에러 메시지의 Property <code>'columns' is missing</code> 이 답
카드 id "5" 중복	복붙	유일하게
<code>type.ts</code> 이름을 전부 <code>XxxProps</code> 로 변경	도메인/props 타입 혼동	되돌리고 <code>CardProps</code> 는 컴포넌트 파일에
<code>function Card({ props }: CardProps)</code>	구조분해 키를 임의로 지음	타입에 있는 키 <code>{ card }</code>
<code>Column.moudle.css</code> 파일명 오타	오타	import가 못 찾음. 파일명 수정
<code>{description && <p></p>}</code>	조건은 맞는데 내용이 비어 있음	<code><p>{description}</p></code>

6. 팁

- TS 에러 읽는 법: `missing`, `not assignable` 앞뒤의 타입 이름과 속성 이름을 찾아 `type.ts` 와 대조.

- 타입 먼저, 데이터 나중: `const x: Board = ...` 로 타입을 먼저 붙이면 에디터가 빠진 필드를 바로 알려준다.
- 작은 것부터: `Card` → `Column` → `Board`. 작은 컴포넌트가 완성돼야 큰 컴포넌트가 그걸 쓸 수 있다.
- 같은 패턴 반복: 세 컴포넌트가 똑같은 모양(별칭 `import` → `Props` 인터페이스 → 구조분해 → `JSX`)이면 읽는 사람이 편하다.

7. 면접 문답 (실제 답변 + 교정)

Q. index를 key로 쓰면 언제 문제가 생기나? 답변: “요소가 삭제되거나 추가될 때 key가 계속 바뀔. 정렬이 이상해질 것 같다.” 교정: 방향은 맞다. 정확히는 key가 자리에 붙어서, 삭제·정렬 시 항목의 로컬 상태가 엉뚱한 항목에 남고 불필요한 리렌더가 생긴다. 모범답안: “key는 배열 렌더링에서 이전 요소와 새 요소를 짝짓는 식별자입니다. index를 쓰면 key가 항목이 아니라 자리에 붙어서, 삭제나 정렬 시 로컬 상태가 엉뚱한 항목에 남고 불필요한 리렌더가 생깁니다. 그래서 안정적인 id를 씁니다.”

Q. props 타입을 `type.ts` 가 아니라 컴포넌트 파일에 둔 이유는? 답변: “데이터 인터페이스와 컴포넌트 인터페이스는 다르다. 컴포넌트엔 나중에 상태 같은 props가 추가될 수 있다.” 교정: 맞다. “응집도”라는 단어를 쓰면 좋다. 그 컴포넌트만 쓰는 타입은 그 파일 옆에.

Q. `{card.description && <p>}` 에서 description이 숫자 0이었다면? 답변: “타입 에러가 먼저 날 것 같고, 아니라면 0이 찍힐 듯.” 교정: 정확. `&&` 는 왼쪽이 falsy면 왼쪽 값을 반환하고 React는 숫자 0을 그린다. 해결은 `!!x &&` 또는 삼항.

Lesson 2 — 카드 CRUD

2단계 교재. 코드는 최종(시그니처 통일 후) 기준.

1. 과제

카드 추가·삭제·제목 수정. 상태는 App의 `useState<Board>` 하나. 상태를 바꾸는 함수는 App에 정의해 `Board` → `Column` → `Card`로 내려보낸다. 불변 업데이트만 사용. 폼은 `AddCardForm`으로 분리.

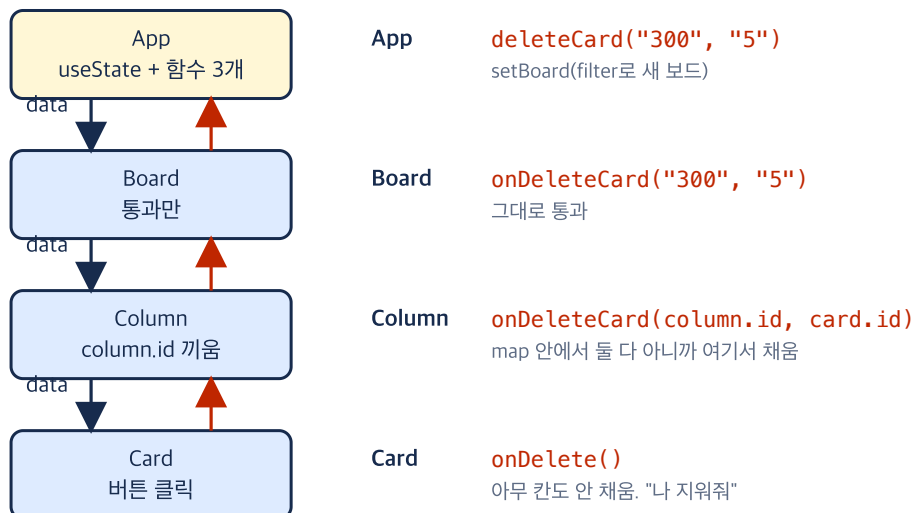
1.5 1단계 대비 나아진 것

	1단계	2단계
데이터	<code>initialBoard</code> 상수를 그대로 그림	<code>useState<Board></code> 상태. <code>initialBoard</code> 는 초기값일 뿐
변경	불가능	추가/삭제/수정. 전부 App의 함수 3개가 불변 업데이트로
흐름	데이터만 아래로	데이터는 아래로 + 사건은 콜백으로 위로
컴포넌트	그리기만	Card는 편집 모드 로컬 상태, <code>AddCardForm</code> 은 제어 컴포넌트
새로 생긴 비용	—	<code>Board</code> 가 쓰지도 않는 콜백 3개를 통과시킴 (<code>prop drilling</code>) → 3단계 동기

2. 전체 그림: 데이터는 아래로, 사건은 위로

2단계: 데이터는 아래로, 사건은 위로. 보고서는 올라가며 한 칸씩 채워진다.

올라가는 보고서



포장지: `onDelete={() => onDeleteCard(column.id, card.id)}` ← Column이 Card를 만들 때 주소를 미리 써서 주는 봉투

- 상태는 App에만 있다. 아래 컴포넌트는 상태를 못 바꾼다. - 아래 컴포넌트는 “이런 일이 일어났다”를 함수 `props`로 위에 보고한다. - 보고서는 올라가는 길에 각 층이 아는 정보(컬럼 id, 카드 id)가 채워진다.

누가 뭘 아는가:

컴포넌트	아는 것	모르는 것
AddCardForm	사용자가 친 제목	자기가 어느 컬럼에 있는지
Card	자기 카드 데이터	자기가 어느 컬럼에 있는지
Column	자기 id, 자기 카드들	사용자가 뭘 칠지
App	전체 보드	어디서 무슨 일이 났는지

3. 최종 코드

3.1 src/App.tsx — 상태와 세 가지 변경 함수

```

import { useState } from "react"
import type { Board as BoardType } from "../type"
import { Board } from "../components/Board"
import { initialBoard } from "../data"

function App() {
  const [board, setBoard] = useState<BoardType>(initialBoard)
  function deleteCard(columnId: string, cardId: string) {
    setBoard({
      ...board,
      columns: board.columns.map((column) => {
        if (column.id !== columnId) return column
        return {
          ...column,
          cards: column.cards.filter((card) => card.id !== card.id),
        }
      }),
    })
  }
  function addCard(columnId: string, title: string) {
    setBoard({
      ...board,
      columns: board.columns.map((column) => {
        if (column.id !== columnId) return column
        return {
          ...column,
          cards: [...column.cards, { id: crypto.randomUUID(), title: title }],
        }
      }),
    })
  }
  function updateTitle(columnId: string, cardId: string, title: string) {
    setBoard({
      ...board,
      columns: board.columns.map((column) => {
        if (column.id !== columnId) return column
        return {
          ...column,
          cards: column.cards.map((card) => {
            if (card.id !== cardId) return card // 관계없는 카드는 그대로
            return { ...card, title } // 해당 카드만 title 바꾼 복사본
          }),
        }
      }),
    })
  }

  return (
    <div>
      <h1>Kanban</h1>
      <Board
        board={board}
        onDeleteCard={deleteCard}
        onAddCard={addCard}
        onUpdateTitle={updateTitle}
      />
    </div>
  )
}

export default App

```

해설 - `useState<BoardType>(initialBoard)` - `initialBoard`는 이제 초기값일 뿐. 진짜 데이터는 `board` 상태. - 세 함수 모두 같은 뼈대: `setBoard({ ...board, columns: board.columns.map(...) })`. “관계없는 컬럼은 그대로 반환, 해당 컬럼만 새 객체로”. - **불변 업데이트 3종**: - 삭제 = `filter` (해당 id가 아닌 것만 남김) - 추가 = `[...column.cards, 새카드]` (기존 뒤에 붙인 새 배열) - 수정 = `map + { ...card, title }` (해당 카드만 복사본, **자리 유지**) - `spread(...)`가 있는 곳마다 “복사본을 만든다”. 원본 `board`, `column`, `cards`는 하나도 안 건드린다. - 새 카드 id는 `crypto.randomUUID()`. - △ `setBoard({ ...board })`가 클로저의 `board`를 읽는다. 지금은 문제없지만 Q3 참고.

3.2 콜백 체인: Board → Column → Card

```
// Board.tsx
import type { Board as BoardData } from "../type"
import { Column } from "../Column"
import styles from "../Board.module.css"
interface BoardProps {
  board: BoardData
  onDeleteCard: (columnId: string, cardId: string) => void
  onAddCard: (columnId: string, title: string) => void
  onUpdateTitle: (columnId: string, cardId: string, title: string) => void
}
export function Board({ board, onDeleteCard, onAddCard, onUpdateTitle }: BoardProps) {
  return (
    <div className={styles.board}>
      {board.columns.map((column) => (
        <Column
          key={column.id}
          column={column}
          onDeleteCard={onDeleteCard}
          onAddCard={onAddCard}
          onUpdateTitle={onUpdateTitle}
        />
      ))}
    </div>
  )
}
```

```
// Column.tsx
import type { Column as ColumnData } from "../type"
import { AddCardForm } from "../AddCardForm"
import { Card } from "../Card"
import styles from "../Column.module.css"
interface ColumnProps {
  column: ColumnData
  onDeleteCard: (columnId: string, cardId: string) => void
  onAddCard: (columnId: string, title: string) => void
  onUpdateTitle: (columnId: string, cardId: string, title: string) => void
}
export function Column({ column, onDeleteCard, onAddCard, onUpdateTitle }: ColumnProps) {
  return (
    <div className={styles.column}>
      <h2 className={styles.title}>{column.title}</h2>
      {column.cards.map((card) => (
        <Card
          key={card.id}
          card={card}
          onDelete={() => onDeleteCard(column.id, card.id)}
          onUpdateTitle={(title) => onUpdateTitle(column.id, card.id, title)}
        />
      ))}
      <AddCardForm onAdd={(title) => onAddCard(column.id, title)} />
    </div>
  )
}
```

해설: 함수는 값이다

```
const onDeleteCard = deleteCard // 새 함수가 아니라 같은 함수에 이름표 하나 더
```

`<Board onDeleteCard={deleteCard} />` 는 Board 안에서 이 대입이 일어난 것. Board 가 `onDeleteCard={onDeleteCard}` 로 넘기면 Column 안에서도 같은 함수.

해설: 포장지(화살표로 감싸기)

```
onDelete={() => onDeleteCard(column.id, cardId)}
```

“cardId 를 받아서, 내 column.id 를 끼워서, 위로 올리는 새 함수”. Card 는 자기 컬럼을 모르니 Column 이 대신 채운다. 두 칸짜리 함수를 한 칸짜리로 바꾸는 어댑터.

코드	무슨 일
<code>onDeleteCard={onDeleteCard}</code>	같은 함수 그대로 통과
<code>onDelete={() => onDeleteCard(column.id, cardId)}</code>	새 함수. 안에서 원래 함수 호출

해설: 시그니처 일관성 (리뷰 지적 → 통일 완료) 1차 리뷰 시점엔 세 콜백이 각각 다른 층에서 id를 끼웠다(`onDeleteCard` 는 Card가 id를 올리고, `onUpdateTitle` 은 Board가 column.id를 끼우는 식). 통일 규칙: - Board 는 전부 통과 (아무것도 모른다) - Column 이 column.id 와 card.id 를 모두 끼운다 (map 안에서 둘 다 안다) - 말단은 payload만 (Card : `onDelete()`, `onUpdateTitle(title)`)

시니어 원칙 세 가지: ① 말단은 모르는 정보를 요구받지 않는다 ② id는 map 도는 곳에서 끼운다 ③ 중간 층은 가능하면 통과 (포장할수록 새 함수가 생겨 9단계 memo에 불리). ②와 ③이 Board 에서 충돌하면 “두 id를 모두 아는 가장 아래 층에서 한 번에”로 푼다.

3.3 src/components/Card.tsx — 편집 모드

```
import { useState } from "react"
import type { Card as CardData } from "../type"
import styles from "./Card.module.css"

interface CardProps {
  card: CardData
  onDelete: () => void
  onUpdateTitle: (title: string) => void
}

export function Card({ card, onDelete, onUpdateTitle }: CardProps) {
  const [isEditing, setIsEditing] = useState<boolean>(false)
  const [text, setText] = useState<string>(card.title)
  function commit() {
    if (!text.trim()) setText(card.title)
    else onUpdateTitle(text.trim())
    setIsEditing(false)
  }
  return (
    <div className={styles.card}>
      {isEditing ? (
        <input
          autoFocus
          value={text}
          onChange={(e) => setText(e.target.value)}
          onKeyDown={(e) => {
            if (e.key === "Enter") {
              commit()
            }
            if (e.key === "Escape") {
              setText(card.title)
              setIsEditing(false)
            }
          }}
          onBlur={commit}
        />
      ) : (
        <h3 className={styles.title} onClick={() => setIsEditing(true)}>
          {card.title}
        </h3>
      )}
      {card.description} && <p className={styles.description}>{card.description}</p>
      <button onClick={onDelete}>삭제</button>
    </div>
  )
}
```

해설 - 제목이 두 개 있다. card.title (부모가 준 진짜 값, <h3>에 보임)과 text (편집용 임시 값, <input>에 보임). - 편집 모드를 닫을 땐 **항상 text를 card.title에 맞춰놓고** 닫는다. 안 그러면 다음에 열 때 낡은 값이 남는다. 확정이 성공하면 부모가 card.title을 바꿔주니 어차피 같아지고, 거부(빈 제목)되면 원복. - onClick은 div가 아니라 <h3>에. div에 두면 input 클릭이 버블링으로 div까지 올라가 편집 모드가 꺼진다. - 토글(!isEditing)이 아니라 setIsEditing(true). 진입과 종료는 다른 이벤트 담당. - autoFocus — 마운트 시 포커스. JSX라 카멜케이스. -

onKeyDown 의 e.key 로 "Enter" / "Escape" 판별. onClick 과 같은 종류의 이벤트 핸들러. - onBlur={commit} — 포커스를 잃으면 확정. Enter와 같은 일이라 commit 함수로 묶어 중복 제거. commit 은 인자를 안 받으니 포장지 없이 그대로 넘겨도 된다. - 확정할 때만 위로 올린다. 키 입력마다 App 상태를 바꾸면 매 타이핑에 보드 전체가 리렌더된다.

3.4 src/components/AddCardForm.tsx — 제어 컴포넌트

```
import { useState } from "react"

interface AddCardFormProps {
  onAdd: (title: string) => void
}

export function AddCardForm({ onAdd }: AddCardFormProps) {
  const [text, setText] = useState("")

  function handleSubmit() {
    if (!text.trim()) return
    onAdd(text.trim())
    setText("")
  }

  return (
    <div>
      <input value={text} onChange={(e) => setText(e.target.value)} />
      <button onClick={handleSubmit}>추가</button>
    </div>
  )
}
```

해설 - 제어 컴포넌트: value={text} + onChange 가 세트. React 상태가 input 값의 유일한 출처. value 만 주고 onChange 를 안 주면 타이핑이 안 되고 경고가 난다. - !text.trim() — “공백 제거 후 비어 있으면 나간다”. !! 로 쓰면 반대가 되어 제목이 있을 때 나가고 없을 때 추가한다(실제로 그렇게 됐었음). - 이 컴포넌트는 “칸반”을 모른다. 글자 받아서 onAdd 로 올릴 뿐. 그래서 재사용 가능. - 개선 여지: <form onSubmit> 으로 바꾸면 input에서 Enter 제출이 되고 시맨틱이 맞다. e.preventDefault() 필요.

4. 흐름 추적 예시: “Done”(id 300)의 “살기”(id 5) 삭제

Card:	onDelete("5")	← 버튼 onClick 포장지가 card.id를 넣음
Column:	onDeleteCard("300", "5")	← column.id 끼움
Board:	onDeleteCard("300", "5")	← 그대로 통과
App:	deleteCard("300", "5")	
	→ setBoard(300번 컬럼의 cards에서 id 5를 filter한 새 보드)	
	→ 리렌더	

5. 대화에서 나온 질문과 답

Q. 상태 만들었는데 그 다음 뭘 해야 하나? 한 기능(삭제)을 끝까지 뚫는다. ① App 에 함수 시그니처만 정의 ② Board 에 넘기고 Props에 타입 추가 ③ Board → Column 통과 ④ Column → Card 에 id 끼워서 ⑤ Card 에 버튼. 여기까지 되면 몸통 채우기.

Q. board.onDeleteCard 라고 썼는데 에러가 난다. 콜백은 데이터 안에 없다. props의 별개 키다. { board, onDeleteCard }: BoardProps 로 둘 다 꺼내야 한다.

Q. `onClick={onDelete}` 가 왜 안 되나? React는 `onClick` 을 마우스 이벤트로 호출한다. `onDelete` 는 문자열을 원한다. `onClick={() => onDelete(card.id)}` 로 포장.

Q. `(title) => onAddCard(column.id, title)` 자체가 이해가 안 된다. §2와 §3.2 해설. “제목 받아서, 내 id 끼워서, 위로 올리는 새 함수”.

Q. `onDeleteCard` 가 곧 `deleteCard` 라는 게 헛갈린다. 함수도 값. `const a = f` 면 `a` 와 `f` 는 같은 함수. `props`로 넘기는 것도 이 대입. 반면 화살표로 감싼 건 새 함수.

Q. `Column` 에서 `card.id` 를 어떻게 끼우나? `Column` 은 `column.id` 만 갖고 있는 거 아닌가? `map` 안에서는 지금 도는 카드가 `card` 변수로 잡혀 있다. `key={card.id}` 에 이미 쓰고 있는 그 변수. 같은 줄의 포장지 안에서도 쓸 수 있다. `column.id` 는 `props`에서, `card.id` 는 `map` 매개변수에서 온다. 출처만 다르지 둘 다 그 줄에서 손에 잡히는 값.

Q. `map`을 쓰니까 되는 건가? 부모 자식이 전부 `map`이라서? `map`은 부모 자식 관계가 아니라 반복문일 뿐. `id`를 끼울 수 있는 진짜 이유는 부모가 자식에게 줄 데이터를 손에 들고 있어서. 카드가 하나뿐이면 `const card = column.cards[0]` 해도 똑같이 `card.id` 를 쓸 수 있다. 질문을 “이 줄에서 어떤 변수가 보이냐”로 바꿔서 생각할 것(클로저).

Q. 데이터가 맨 위(App)에 있으니 App에서 다 잡을 수도 있는 거 아닌가? App은 데이터 전부를 갖지만 “어느 카드가 눌렸는지”는 모른다. 그 정보는 그리는 순간(`map`)이나 놀리는 순간(`leaf`)에만 생긴다. App이 다 잡으려면 컴포넌트를 안 나누고 App이 직접 이중 `map`을 돌아야 한다. 컴포넌트를 나눈다 = “어느 항목인지 아는 지점”을 아래로 내려보낸다.

Q. 카드는 자기 `id`만 처리해야 하는 거 아닌가? 왜 `Column`이 `card.id`를 대신 넣나? 그 직관도 맞는 방식이다(`Card`가 `onDelete(card.id)` 로 올리기). 다만 `card` 객체는 원래 `Column` 이 갖고 있던 걸 `Card` 에 건넨 것이라 둘 다 안다. 누가 쓰느냐는 설계 선택. 봉투 비유: `Card` 가 클릭 시 주소를 적어 부치느냐, `Column` 이 만들 때 주소를 미리 써서 주느냐. 후자를 고른 이유는 `Card` 가 “지워줘” 한 마디(`() => void`)만 하면 되어 더 단순해서. HTML `<button onClick={() => remove(5)}>` 에서 버튼이 5를 모르는 것과 같다.

Q. `onUpdateTitle` 엔 왜 `cardId`를 안 올리나? `Column` 이 `map` 안에서 `card.id` 를 이미 알기 때문에 대신 끼울 수 있다. 삭제도 똑같이 할 수 있었다. 어느 쪽이든 되지만 섞지 말 것.

Q. 수정 기능은 어느 파일부터? 잘 모르는 쪽부터. 체인은 아니까 새 개념(편집 모드)이 있는 `Card` 부터. 로컬 상태로 토글만 먼저 만들고, `CardProps` 에 콜백을 추가하면 TS 빨간 줄이 `Column` → `Board` → `App` 순으로 다음 할 일을 가리킨다.

Q. `blur`가 뭔가? 포커스를 잃는 것. 편집 중 다른 곳을 클릭하면 저장하고 닫히게 하려고 `onBlur` 에 `commit`.

Q. “text가 안 돌아간다”는 게 무슨 뜻? §3.3 해설. `card.title` 과 `text` 가 따로 놀아서, Enter로 닫아도 `text` 에 남은 값이 남는 문제.

6. 삽질 기록

증상	원인	해결
<code>board.onDeleteCard</code> 에러	콜백을 데이터에서 찾을 수 없음	<code>props</code> 에서 구조분해
<code>onClick={onDelete}</code> 타입 에러	이벤트가 인자로 들어감	화살표 포장
추가 버튼이 반대로 동작	<code>!!text.trim()</code> 조건 반전	<code>!text.trim()</code>
편집 모드 진입 즉시 꺼짐, 포커스 불가	<code>div</code> 의 <code>onClick</code> 이 <code>input</code> 클릭 버블링을 받아 토글	<code>h3</code> 로 이동, <code>setIsEditing(true)</code>
수정하면 카드가 맨 아래로 가고 <code>description</code> 사라짐	<code>filter + append</code> = 삭제 후 추가	<code>map + { ...card, title }</code>
<code>updateTitle</code> 몸통에서 <code>cards:</code> 뒤가 빈	문법 에러 <code>Expression expected</code>	위 <code>map</code> 패턴

7. 팁

- 한 기능을 끝까지: 삭제 하나를 App→Card까지 완주하면 나머지는 같은 패턴 반복.
- TS 에러 = 할 일 목록: 아래에서 위로 갈 때 빨간 줄이 다음 파일을 가리킨다.
- 조건문은 소리 내 읽기: `!`가 들어간 조건은 “이게 참일 때 뭘 하나”를 말로 옮겨 반대로 쓴 걸 잡는다.
- 불변 업데이트 3종 암기: 삭제 `filter`, 추가 `[...arr, x]`, 수정 `map + spread`.
- 부모에 클릭 핸들러 + 자식이 인터랙티브 = 버블링 의심.
- 포매팅은 도구에: `pnpm format` (Prettier).

8. 면접 문답 (실제 답변 + 교정)

Q1. 왜 상태를 직접 수정하면 안 되는가? 답변: “원본을 바꾸면 이전이랑 같은 객체라고 판단해서 React가 바뀐 줄 모름.” 교정: 맞다. 정확히는 `Object.is`(이전, 새것)로 참조를 비교하고 내용은 안 본다. `push`는 참조가 같아 리렌더를 건너뛰고, `spread`는 참조가 달라 감지된다. 모범답안: “React는 상태를 참조 동등성으로 비교하기 때문에, 원본을 변경하면 참조가 같아 변경을 감지하지 못합니다. 그래서 `spread`나 `map/filter`로 새 객체를 만듭니다.”

Q2. 제어 컴포넌트가 뭐가? 답변: “폼 요소 값을 컴포넌트 상태로 관리하고 동기화. `onChange`를 `value`와 연결해야 칠 때마다 리렌더되어 `value`가 갱신됨. 성능 문제 있을 수도.” 교정: 맞다. 핵심 단어는 **단일 진실 원천**. DOM `input`이 값을 갖지 않고 React 상태만 값을 가진다. `value`만 있고 `onChange`가 없으면 타이핑이 씹히고 경고. 성능 우려는 맞지만 그래서 `AddCardForm`을 분리한 것 — 리렌더 범위가 그 품뿐.

Q3. `setBoard(prev => ...)`는 언제 필요한가? 답변: 모름.

useState는 '줄 세우기'. 같은 이벤트에서 addCard를 두 번 부르면?



적용 시점은 둘 다 '다음 렌더'. 차이는 뭘 읽느냐: 클로저의 낡은 값 vs React가 넣어주는 최신 값.

값 형태 vs 함수 형태 setter

설명: useState는 그 자리에서 바꾸는 게 아니라 “다음 렌더에 적용해줘”라고 줄을 세우는 것(배칭). 값/함수 형태 모두 적용 시점은 같다. 차이는 뭘 읽느냐.

	줄에 세우는 것	계산 시점	읽는 이전 상태
setBoard({ ...board })	완성된 값	지금, 내가	클로저의 board (낡을 수 있음)
setBoard(prev => ...)	레시피	나중에, React가	줄 앞 결과 (항상 최신)

한 이벤트에서 addCard를 두 번 부르면 값 형태는 둘 다 낡은 board를 읽어 하나가 사라진다. 함수 형태는 두 번째의 prev가 첫 번째 결과라 둘 다 남는다. 규칙: **새 상태가 이전 상태에 의존하면 함수형**. 지금 안 고치는 이유는 3단계 useReducer가 항상 최신 상태를 받아 구조적으로 해결되기 때문. 7단계 await 뒤 갱신에서 실제로 만난다. 동기/비동기 문제가 아니다. 둘 다 “나중에 적용”이고, 함수형은 “읽기”까지 나중에 미루는 것.

Lesson 3 — useReducer + Context 리팩터

3단계 교재. 동작은 2단계와 똑같다. 구조만 바뀐다.

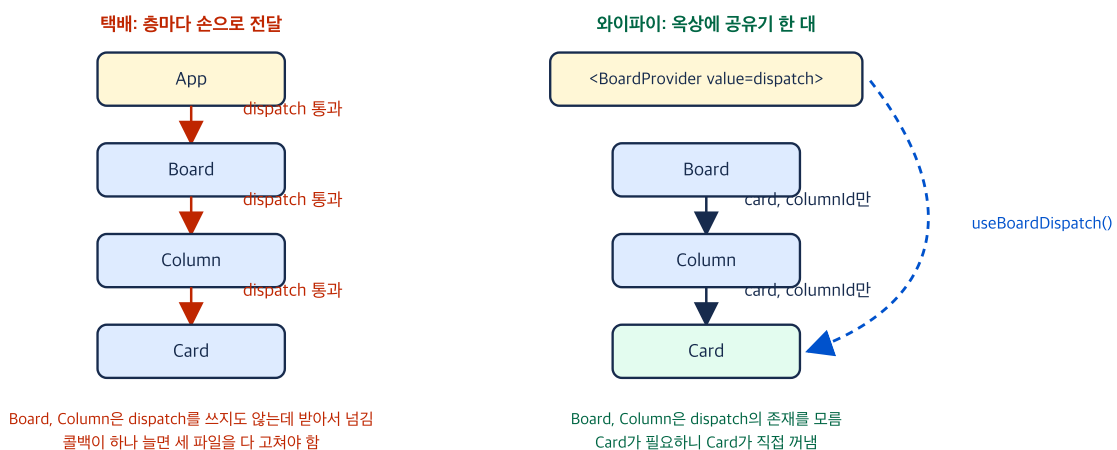
1. 과제

2단계의 prop drilling(Board 가 쓰지도 않는 콜백 3개를 통과)을 없앴다. 상태 변경 로직을 App 에서 리듀서(순수 함수)로 옮기고, dispatch 를 Context로 배달해 말단이 직접 부른다. 리팩터 = 겉보기 동작 유지 + 내부 구조 변경. 2단계 완료 조건 이 끝까지 전부 통과해야 한다.

1.5 2단계 대비 나아진 것

	2단계	3단계
상태 변경 로직	App 안의 함수 3개 (컴포넌트에 묶임)	boardReducer 순수 함수 (React 없이 호출·테스트 가능)
변경 통로	콜백 3개를 각각 내려보냄	dispatch 하나. 액션 객체만 다름
중간 컴포넌트	Board 가 콜백 3개를 통과만 시킴	Board, Column 은 콜백의 존재를 모름
말단	Card 는 onDelete() 호출	Card 가 dispatch({ type: "deleteCard", ... }) 직접
함수형 setter 문제	setBoard({ ...board }) 가 낡은 값 읽을 위험	리듀서가 항상 최신 state 를 받아 구조적으로 해결
액션 추가 시	함수 추가 + 세 파일의 props 수정	액션 타입 추가 + case 추가. 빼먹으면 never 가 컴파일 에러
새로 생긴 비용	—	Card / AddCardForm 이 columnId 를 알아야 함. 파일 3개 추가. 리렌더 범위는 아직 개선 안 됨(9단계)

prop drilling (2단계) vs Context (3단계)



2. 전체 그림

2단계 (before)

App(useState + 함수3개) —board, 콜백3개→ Board —콜백3개→ Column —포장지→ Card
(안 씀, 통과) (id 끼움)

3단계 (after)

```
App
├ <BoardProvider>
│   └ Board: useBoard()
│       └ Column
│           └ Card: useBoardDispatch()
│               └ AddCardForm: useBoardDispatch()
└ useReducer가 여기. board/dispatch를 Context에 실음
    └ 와이파이로 board를 잡음. props 없음
        └ card, columnId만 내려줌. 콜백 모름
            └ dispatch({ type: "deleteCard", ... })
            └ dispatch({ type: "addCard", ... })
```

리듀서와 Context는 독립이다

- **리듀서**: “이전 상태 + 무슨 일(액션) → 다음 상태”를 계산하는 **순수 함수**. 상태를 어떻게 바꾸나를 한 곳에 모은다.
- **Context**: props를 거치지 않고 위에서 넣은 값을 아래 어디서든 꺼내는 **배달 통로**. 상태를 관리하지 않는다.
- 둘을 같이 쓰면 2단계 콜백 체인이 통째로 사라진다. 그래서 순서가 “리듀서 먼저, Context 나중”.

파일 구조

src/	
type.ts	도메인 타입
data.ts	초기 데이터
boardReducer.ts	BoardAction 타입 + boardReducer (순수 함수)
BoardContext.ts	Context 객체 2개 (컴포넌트 없음)
BoardProvider.tsx	Provider 컴포넌트 (useReducer가 여기 삶)
useBoard.ts	useBoard / useBoardDispatch 혹은
App.tsx	Provider로 감싸기만
components/	Board, Column, Card, AddCardForm

세 파일로 나눈 이유: Vite Fast Refresh는 “컴포넌트만 export하는 파일”에서만 동작. Context 객체나 훅이 섞이면 lint 경고.

3. 최종 코드

3.1 src/boardReducer.ts

```

import type { Board } from "./type"

export type BoardAction =
  | { type: "addCard"; columnId: string; title: string; cardId: string }
  | { type: "deleteCard"; columnId: string; cardId: string }
  | { type: "updateCardTitle"; columnId: string; cardId: string; title: string }

export function boardReducer(state: Board, action: BoardAction): Board {
  switch (action.type) {
    case "addCard": {
      // App.addCard 몸통. board → state, setBoard({...}) → return {...}
      return {
        ...state,
        columns: state.columns.map((column) => {
          if (column.id !== action.columnId) return column
          return {
            ...column,
            cards: [...column.cards, { id: action.cardId, title: action.title }],
          }
        }),
      }
    }
    case "deleteCard": {
      return {
        ...state,
        columns: state.columns.map((column) => {
          if (column.id !== action.columnId) return column
          return {
            ...column,
            cards: column.cards.filter((card) => action.cardId !== card.id),
          }
        }),
      }
    }
    case "updateCardTitle": {
      return {
        ...state,
        columns: state.columns.map((column) => {
          if (column.id !== action.columnId) return column
          return {
            ...column,
            cards: column.cards.map((card) => {
              if (card.id !== action.cardId) return card
              return { ...card, title: action.title }
            }),
          }
        }),
      }
    }
    default: {
      const exhaustive: never = action
      return exhaustive
    }
  }
}

```

해설 - BoardAction은 **discriminated union**. |로 “셋 중 하나”, type 필드로 구분. switch (action.type)의 각 case 안에서 TS가 action의 타입을 좁혀준다. case "addCard" 안에서 action.cardId는 되고 action.title도 되지만, case "deleteCard" 안에서 action.title은 빨간 줄. - 각 case 몸통은 2단계 App의 세 함수에서 그대로 옮긴 것. 바뀐 건 board → state, setBoard({...}) → return {...} 둘뿐. - **순수 함수**. 같은 (state, action)이면 항상 같은 결과. 그래서 crypto.randomUUID()를 안에 두지 않고 action.cardId로 받는다. 이유: ① 테스트에서

`expect(reducer(s, a)).toEqual(기대값)` 을 쓸 수 있다 ② `React.StrictMode`는 개발 중 리듀서를 두 번 호출하는데, 부수효과가 있으면 두 번 일어난다. - `default`의 `const exhaustive: never = action` — “여기 오는 액션은 없어야 한다”는 선언. 나중에 `BoardAction`에 `moveCard`를 추가하고 case를 안 만들면 `action`이 `never`가 아니게 되어 컴파일 에러. “액션 추가하고 리듀서 수정 깜빡함”을 실행 전에 잡는다. `return exhaustive`는 안 쓰는 변수 에러를 피하려는 것 (`never`는 어떤 타입에도 대입 가능).

3.2 `src/BoardContext.ts` / `src/BoardProvider.tsx` / `src/useBoard.ts`

```
// BoardContext.ts
import { createContext, type Dispatch } from "react"
import type { Board } from "../type"
import type { BoardAction } from "../boardReducer"

// 와이파이 공유기 두 대. 하나는 상태용, 하나는 dispatch용.
// 둘을 나누는 이유: board는 매 변경마다 새 객체라 자주 바뀌고, dispatch는 절대 안 바뀐다.
// 한 Context에 넣으면 dispatch만 쓰는 컴포넌트도 board가 바뀔 때마다 리렌더된다.
export const BoardContext = createContext<Board | null>(null)
export const BoardDispatchContext = createContext<Dispatch<BoardAction> | null>(null)
```

```
// BoardProvider.tsx
import { useReducer, type ReactNode } from "react"
import { BoardContext, BoardDispatchContext } from "../BoardContext"
import { boardReducer } from "../boardReducer"
import { initialBoard } from "../data"

// 공유기를 켜는 컴포넌트. 상태는 여기 useReducer에 산다.
export function BoardProvider({ children }: { children: ReactNode }) {
  const [board, dispatch] = useReducer(boardReducer, initialBoard)
  return (
    <BoardContext.Provider value={board}>
      <BoardDispatchContext.Provider value={dispatch}>{children}</BoardDispatchContext.Provider>
    </BoardContext.Provider>
  )
}
```

```
// useBoard.ts
import { useContext } from "react"
import { BoardContext, BoardDispatchContext } from "../BoardContext"

// 와이파이 잡는 훅. Provider 밖에서 쓰면 null이라 바로 터뜨린다.
export function useBoard() {
  const board = useContext(BoardContext)
  if (board === null) throw new Error("useBoard는 BoardProvider 안에서만 쓸 수 있다")
  return board
}

export function useBoardDispatch() {
  const boardDispatch = useContext(BoardDispatchContext)
  if (boardDispatch === null)
    throw new Error("useBoardDispatch는 BoardProvider 안에서만 쓸 수 있다")
  return boardDispatch
}
```

해설 - `createContext<Board | null>(null)` — 공유기. Provider로 안 감쌌을 때 기본값이 `null`. 훅에서 `null` 검사로 “Provider 없이 썼다”를 조용히 넘기지 않고 바로 터뜨린다. - `BoardProvider`는 평범한 컴포넌트. 2단계 App에 있던 `useReducer`가 여기로 왔다. 상태는 Context가 아니라 이 컴포넌트의 `useReducer`에 산다. `children`을 Provider 안에 그대로 그린다. - `useReducer(boardReducer, initialBoard)`는 `[board, dispatch]`. `useState`와 모양이 같고,

바꾸는 함수에 **새 값** 대신 **액션**을 넣는다. `dispatch(액션)` → React가 `boardReducer(현재 board, 액션)` 호출 → 결과를 새 board로. - **Context를 둘로 나눈 이유: 리렌더 범위.** Context 값이 바뀌면 그걸 구독하는 컴포넌트가 전부 리렌더. `board`는 카드 하나만 바뀌어도 새 객체(매번 바뀜), `dispatch`는 앱이 사는 동안 불변. 한 Context에 넣으면 `dispatch`만 쓰는 `AddCardForm`도 보드 변경마다 리렌더. 나누면 안 흔들린다. (지금 앱에선 `Board→Column→Card`가 어차피 다 리렌더돼 효과가 안 보인다. 9단계 `memo`에서 체감.) - `Dispatch<BoardAction> = (action: BoardAction) => void`. React 제공 타입.

3.3 src/App.tsx

```
import { BoardProvider } from "../BoardProvider"
import { Board } from "../components/Board"

function App() {
  return (
    <BoardProvider>
      <div>
        <h1>Kanban</h1>
        <Board />
      </div>
    </BoardProvider>
  )
}

export default App
```

해설 - App은 Provider로 감싸기만. 상태도, 함수도 없다. - App은 `useBoard()`를 못 쓴다. 자기가 Provider를 그리는 쪽이라 그 밖에 있다. 와이파이의 공유기 아래층부터 잡힌다.

3.4 src/components/Board.tsx / Column.tsx

```
// Board.tsx
import { Column } from "../Column"
import styles from "../Board.module.css"
import { useBoard } from "../useBoard"

export function Board() {
  const board = useBoard()
  return (
    <div className={styles.board}>
      {board.columns.map((column) => (
        <Column key={column.id} column={column} />
      ))}
    </div>
  )
}
```

```
// Column.tsx
import type { Column as ColumnData } from "../type"
import { AddCardForm } from "../AddCardForm"
import { Card } from "../Card"
import styles from "../Column.module.css"
interface ColumnProps {
  column: ColumnData
}
export function Column({ column }: ColumnProps) {
  return (
    <div className={styles.column}>
      <h2 className={styles.title}>{column.title}</h2>
      {column.cards.map((card) => (
        <Card key={card.id} card={card} columnId={column.id} />
      ))}
      <AddCardForm columnId={column.id} />
    </div>
  )
}
```

해설 - Board 는 props가 없다. useBoard() 로 잡는다. - Column 은 card 와 columnId 만 내려준다. 콜백은 모른다. - columnId={column.id} — 2단계의 “누가 id를 끼우나”가 이 형태로 돌아왔다. Card 가 dispatch 를 직접 부르니 자기 컬럼 id를 알아야 하고, 그걸 Column 이 props로 준다.

3.5 src/components/Card.tsx / AddCardForm.tsx

```

// Card.tsx
import { useState } from "react"
import type { Card as CardData } from "../type"
import styles from "./Card.module.css"
import { useBoardDispatch } from "../useBoard"

interface CardProps {
  card: CardData
  columnId: string
}

export function Card({ card, columnId }: CardProps) {
  const [isEditing, setIsEditing] = useState<boolean>(false)
  const [text, setText] = useState<string>(card.title)
  const dispatch = useBoardDispatch()
  function commit() {
    if (!text.trim()) setText(card.title)
    else dispatch({ type: "updateCardTitle", columnId, cardId: card.id, title: text.trim() })
    setIsEditing(false)
  }
  return (
    <div className={styles.card}>
      {isEditing ? (
        <input
          autoFocus
          value={text}
          onChange={(e) => setText(e.target.value)}
          onKeyDown={(e) => {
            if (e.key === "Enter") {
              commit()
            }
            if (e.key === "Escape") {
              setText(card.title)
              setIsEditing(false)
            }
          }}
          onBlur={commit}
        />
      ) : (
        <h3 className={styles.title} onClick={() => setIsEditing(true)}>
          {card.title}
        </h3>
      )}
      {card.description} && <p className={styles.description}>{card.description}</p>
      <button onClick={() => dispatch({ type: "deleteCard", columnId, cardId: card.id })}>
        삭제
      </button>
    </div>
  )
}

```

```
// AddCardForm.tsx
import { useState } from "react"
import { useBoardDispatch } from "../useBoard"

interface AddCardFormProps {
  columnId: string
}

export function AddCardForm({ columnId }: AddCardFormProps) {
  const [text, setText] = useState("")
  const dispatch = useBoardDispatch()

  function handleSubmit() {
    if (!text.trim()) return
    dispatch({ type: "addCard", columnId, title: text.trim(), cardId: crypto.randomUUID() })
    setText("")
  }

  return (
    <div>
      <input value={text} onChange={(e) => setText(e.target.value)} />
      <button onClick={handleSubmit}>추가</button>
    </div>
  )
}
```

해설 - `const dispatch = useBoardDispatch()` — 와이파이 잡기. - 2단계에서 `onUpdateTitle(text.trim())` 한 마디 하던 자리에서, 이제 “어느 컬럼, 어느 카드, 무슨 제목”을 액션 객체 하나에 담아 던진다. `columnId`는 props에서, `card.id`는 원래 갖고 있던 것. - 삭제 버튼은 `dispatch`에 인자를 넣어야 하니 포장지 `() => ...`가 다시 붙는다. - `AddCardForm`이 `cardId: crypto.randomUUID()`를 만든다. 부수효과는 `dispatch`하는 쪽에서. 이제 이 컴포넌트는 “칸 반 컬럼”을 안다(`columnId`). 2단계의 “칸반을 모르는 폼”과 트레이드오프.

4. 리듀서는 그냥 함수다 — React 없이 돌려보기

Node에서 리듀서를 직접 호출한 결과:

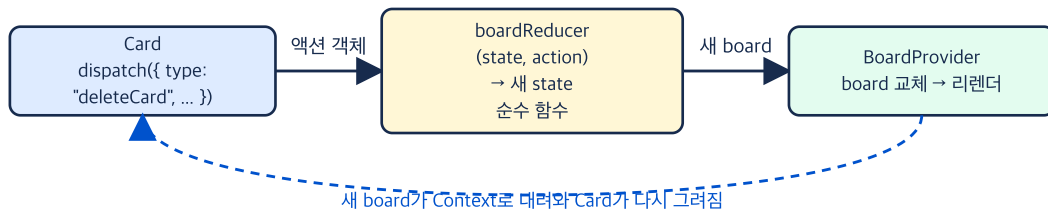
```
const s1 = boardReducer(initialBoard, { type: "deleteCard", columnId: "100", cardId: "1" })
const s2 = boardReducer(s1, { type: "addCard", columnId: "300", cardId: "x", title: "빨래하기" })
const s3 = boardReducer(s2, { type: "updateCardTitle", columnId: "200", cardId: "3", title: "놀이!!" })
```

0. 시작	To Do: [공부하기, 잠자기]	In Progress: [놀이, 먹기]	Done: [살기, 태어나기]
1. 공부하기 삭제	To Do: [잠자기]	In Progress: [놀이, 먹기]	Done: [살기, 태어나기]
2. Done에 추가	To Do: [잠자기]	In Progress: [놀이, 먹기]	Done: [살기, 태어나기, 빨래하기]
3. 놀기 수정	To Do: [잠자기]	In Progress: [놀이!!, 먹기]	Done: [살기, 태어나기, 빨래하기]
원본은 그대로?	To Do: [공부하기, 잠자기]	...	(initialBoard 변화 없음)
s1 === s2 ?	false		(매번 새 객체)

- 보드 넣고 액션 넣으면 새 보드. React도 훅도 없이 돌아간다.
- 이전 결과를 다음 입력으로 넣으면 이어진다. 앱에서 클릭 세 번 = React가 이걸 세 번 해주는 것.
- 원본은 안 건드린다. 참조가 매번 다르다(2단계 Q1).
- 8단계 테스트는 이 코드를 `expect(...)`로 감싸는 것.

useReducer의 순환: dispatch → 리듀서 → 새 상태 → 리렌더

React가 뒤에서 하는 일: board = boardReducer(board, action)



리듀서는 React 없이도 호출 가능 → Node에서 직접 돌려 확인함 (3장 §4). 8단계 테스트의 대상.

useReducer 순환

useReducer 가 하는 일 한 줄: 현재 보드를 기억하고 있다가, dispatch(액션) 이 오면 boardReducer(현재보드, 액션) 을 호출하고, 결과를 새 현재 보드로 바꿔치기한 뒤 리렌더.

5. 대화에서 나온 질문과 답

Q. 리듀서랑 Context가 뭔가? §2. 리듀서 = 2단계 App 의 세 함수를 하나로 합친 순수 함수. Context = props 안 거치는 배달 통로. 서로 독립.

Q. useReducer 가 무슨 뜻인지 모르겠다.

	useState	useReducer
돌려주는 것	[값, 바꾸는 함수]	[값, 바꾸는 함수] 똑같음
바꾸는 함수에 넣는 것	새 값 setBoard(새보드)	액션 dispatch({ type, ... })
새 값을 누가 계산	내가	boardReducer 가

“useState 인데, 새 값을 내가 만드는 대신 무슨 일이지만 말하면 리듀서가 만들어주는 버전.”

Q. Context 역할이 뭔가? “props를 거치지 않고, 위에서 한 번 넣어둔 값을 아래 어디서든 꺼내 쓰게 해주는 통로.” 상태 관리는 안 한다. 택배(props) vs 건물 와이파이(Context). 바로 아래 자식한테 주는 건 그냥 props가 맞다.

Q. 전역 상태는 아닌 거지? 아니다. ① Provider 서브트리 안에서만 잡힌다 ② Context 자체는 상태를 안 갖는다(상태는 BoardProvider 의 useReducer 에). Redux/Zustand는 React 밖 store에 상태가 있고 앱 어디서든 접근.

	Context	Redux / Zustand
상태가 사는 곳	Provider 컴포넌트 안	React 밖 store
접근 범위	Provider 서브트리	앱 어디서든
본질	값 배달(의존성 주입)	저장소

Q. 리듀서 틀, Context 틀 잡아달라. §3.1, §3.2. 몸통은 사용자가 2단계 코드에서 옮김.

Q. App 에서 board , dispatch 가 빨간 줄인데 어떻게 푸나? App 에 가져오는 게 아니라 App 이 더 이상 아무것도 안 넘기면 풀린다. 그러면 빨간 줄이 Board 로 넘어가고, 거기서 useBoard() . 빨간 줄을 위에서 아래로 밀어낸다.

6. 삼질 기록

증상	원인	해결
<code>_exhaustive</code> never read	tsconfig <code>noUnusedLocals</code>	<code>return exhaustive</code> (never는 어디든 대입 가능)
<code>useReducer</code> 썼는데 import 안 바꿈, <code>setBoard</code> 이름으로 <code>dispatch</code> 쓰려 함	개념 전환 중	<code>[board, dispatch] = useReducer(...)</code>
lint: Fast refresh only works when a file only exports components	혹/Context 객체가 컴포넌트와 한 파일	<code>BoardContext.ts</code> / <code>BoardProvider.tsx</code> / <code>useBoard.ts</code> 셋으로 분리
dev 서버 꺼짐	터미널 종료	<code>.claude/launch.json</code> 으로 재시작

7. 팁

- 리팩터는 한 번에 하나. 리듀서 도입(콜백 유지) → 동작 확인 → Context 도입. 둘을 동시에 바꾸면 뭐가 깨졌는지 못 찾는다.
- 빨간 줄 밀어내기. App 에서 props를 지우면 에러가 Board 로, Board 에서 지우면 Column 으로. TS가 다음 파일을 가리킨다.
- 리듀서 디버깅은 Node에서. React 없이 `boardReducer(state, action)` 호출해 결과를 찍어보면 가장 빠르다.
- Context 값은 “안 바뀌는 것”과 “자주 바뀌는 것”을 나눠 싼다.
- 순수성 체크: 리듀서 안에 `Math.random`, `Date.now`, `crypto`, `fetch`, `console.log` 가 있으면 밖으로.

8. 면접 문답 (실제 답변 + 교정)

- Q1. `useState` 로 충분한데 왜 `useReducer` 를 쓰나? 언제 갈아타나? 답변: “상태 변경 로직이 복잡할 때.” 교정: 맞지만 “복잡”을 구체화. ① 여러 액션이 같은 상태를 건드릴 때 ② 다음 상태가 이전 상태에 의존할 때(함수형 setter 문제 소멸) ③ 로직을 컴포넌트 밖으로 빼서 테스트하고 싶을 때. 갈아타는 신호: `setX` 가 3개 이상이고 같은 객체를 만지기 시작할 때.
- Q2. 상태와 `dispatch`를 왜 별도 Context로 나누나? 답변: “쓰는 곳이 달라서?” 교정: 이유는 리렌더 범위. Context 값이 바뀌면 구독자 전부 리렌더. `board` 는 매번 새 객체, `dispatch` 는 불변. 합치면 `dispatch` 만 쓰는 컴포넌트도 `board` 변경마다 리렌더. 현재 앱은 어차피 다 리렌더돼 안 보이고, 9단계 memo 에서 체감.
- Q3. Context는 전역 상태 라이브러리와 뭐가 다른가? 답변: “자기 자식 트리에서만 쓸 수 있고, 상태가 외부 저장소에 있는 게 아니라 reducer에 있다. 중간의 의존성 주입을 편하게 해주는 것.” 교정: 정확. 모범답안: “Context는 저장소가 아니라 Provider 서브트리 안에서 값을 배달하는 의존성 주입 도구고, 상태 자체는 `useReducer`가 가집니다. 한계는 값이 바뀌면 구독자가 전부 리렌더되고 선택적 구독이 안 된다는 점입니다.”
- Q4. 리듀서가 순수해야 하는 이유는? 답변: “같은 입력에 같은 값 나와야 해서?” 교정: 핵심 맞음. 따라오는 것: ① 테스트에서 `toEqual` 로 기대값 비교 가능 ② StrictMode가 리듀서를 두 번 호출해도 안전. 그래서 `crypto.randomUUID()` 를 `dispatch` 쪽으로 뺐다.

Q5. exhaustive check(never)가 뭘 막아주나? 답변: 모름. 설명: 액션 타입을 추가하고 case를 빼먹었을 때, default의 action이 never가 아니게 되어 컴파일 에러. “액션 추가 후 리듀서 수정 누락”을 실행 전에 잡는다. 5단계 moveCard 추가 시 체감.

부록 A — 핵심 개념 한눈에

개념	한 줄	어디서
옵셔널 ?	“비어 있을 수 있음”이 아니라 “없을 수 있음(undefined)”. 컬렉션은 [] 로 필수.	1장
도메인 타입 vs Props 타입	데이터 모양(<code>type.ts</code>) vs 컴포넌트 입력 계약(컴포넌트 파일 안). props엔 콜백도 들어간다.	1장
매개변수 구조분해	<code>{ card }</code> : <code>CardProps</code> 의 종괄호 안 이름은 타입에 실제 있는 키.	1장
key	map이 반환하는 요소에, 안정적인 id로. props가 아님. index면 “자리”에 붙어 상태가 엉뚱한 항목에 남음.	1장
&& 조건부 렌더링	왼쪽이 falsy면 왼쪽 값 반환. 0 은 그려진다.	1장
flex 방향	자식이 아니라 부모 컨테이너에.	1장
단방향 데이터 흐름	데이터는 props로 아래로, 사건은 콜백으로 위로. 상태는 한 곳.	2장
함수는 값	<code>onX={f}</code> 는 같은 함수에 이름표 추가. 화살표로 감싸면 새 함수.	2장
포장지 패턴	<code>onClick={() => f(id)}</code> (인자 넣기), <code>onX={ (a) => g(myId, a) }</code> (내 정보 끼우기).	2장
콜백 시그니처 원칙	말단은 모르는 정보 요구 안 받음 / id는 map 도는 곳에서 / 중간층은 통과.	2장
불변 업데이트 3종	삭제 <code>filter</code> , 추가 <code>[...arr, x]</code> , 수정 <code>map + { ...x, k }</code> . React는 참조(<code>Object.is</code>)로 변경 감지.	2장
제어 컴포넌트	<code>value + onChange</code> 세트. React 상태가 단일 진실 원천.	2장
이벤트 버블링	자식 이벤트가 부모로 전파. 부모 <code>onClick</code> + 자식 <code>input</code> = 의심.	2장
setState는 줄 세우기	값 형태는 클로저의 낡은 값으로 지금 계산, 함수형은 React가 최신 상태로 나중에 계산. 의존하면 함수형.	2장
리듀서	<code>(state, action) => newState</code> 순수 함수. React 없이 호출 가능.	3장
useReducer	useState인데 새 값 대신 액션을 넣고 리듀서가 계산.	3장

개념	한 줄	어디서
discriminated union + never	type 으로 좁히기. default 의 never 로 case 누락을 컴파일 타임에.	3장
리듀서 순수성	같은 입력 같은 출력. 랜덤/시간/fetch는 dispatch 쪽에서. StrictMode 2회 호출 안전.	3장
Context	props 안 거치는 배달 통로(의존성 주입). 상태 관리 아님. Provider 서브트리 범위. 전역 아님.	3장
Context 분리	자주 바뀌는 값(board)과 불변(dispatch)을 나눠 리렌더 범위 축소.	3장
Fast Refresh	컴포넌트만 export하는 파일에서만. 훅/Context 객체는 파일 분리.	3장

부록 B — 면접 질문 모음 (답할 수 있어야 하는 것)

1단계

1. key는 왜 필요하고, index를 key로 쓰면 언제 문제가 되는가?
2. 정규화된 상태와 중첩 상태의 트레이드오프는? 이 프로젝트는 뭘 택했고 왜?
3. `interface`와 `type`의 차이, props 타입엔 뭘 쓰는가?
4. 컴포넌트를 나누는 기준은?
5. 옵셔널 필드는 언제 쓰는가?

2단계

1. 왜 상태를 직접 수정하면 안 되는가? React는 어떻게 “바뀌었다”를 판단하는가?
2. 제어 컴포넌트와 비제어 컴포넌트의 차이는? 왜 제어를 썼는가?
3. 편집 중 텍스트를 로컬 상태로 두고 확정할 때만 올리는 것과 키 입력마다 올리는 것의 차이는?
4. `AddCardForm`을 분리한 이유는?
5. `setState(prev => ...)`는 언제 필요한가?
6. 이벤트 버블링이 뭔가?
7. 콜백에서 id를 누가 끼우는 게 좋은가? 그 이유는?

3단계

1. `useState`로 충분한데 왜 `useReducer`를 쓰나? 언제 갈아타나?
2. 상태와 dispatch를 왜 별도 Context로 나누나?
3. Context는 전역 상태 라이브러리와 뭐가 다른가? 한계는?
4. 리듀서가 순수해야 하는 이유는? 테스트와 어떤 관계인가?
5. discriminated union의 exhaustive check가 뭘 막아주나?

부록 C — 작업 습관 (팁 모음)

- TS 에러 읽는 법: `missing`, `not assignable` 앞뒤의 타입 이름·속성 이름을 찾아 타입 정의와 대조.
- 타입 먼저, 데이터 나중. `const x: Board = ...` 로 붙이면 빠진 필드를 에디터가 알려준다.
- 작은 것부터. Card → Column → Board.
- 한 기능을 끝까지. 삭제 하나를 App→Card까지 완주하면 나머지는 반복.
- 잘 모르는 쪽부터 시작. 체인을 알면 말단(Card)부터, 말단을 알면 App부터.
- TS 에러 = 할 일 목록. props를 지우거나 추가하면 빨간 줄이 다음 파일을 가리킨다.
- 조건문은 소리 내 읽기. `!` 가 들어간 조건은 “참일 때 뭘 하나”를 말로.
- 리팩터는 한 번에 하나. 리듀서 도입 → 확인 → Context 도입.
- 리듀서 디버깅은 Node에서. React 없이 호출해 출력을 본다.
- 포매팅은 도구에. `pnpm format`.